

Python Cheatsheet

MATH60033A · Session 05 · Regularisation

Thierry Warin, PhD · HEC Montréal · A2026

Everything you need to type, in one place. The first three sections are the same every week; the last one is the Python from session 4.

Starting Python in VS Codium

Open the course folder once — **File > Open Folder**, then pick the repository. Everything below assumes you are inside it.

Open a terminal with **View > Terminal** (Ctrl + ` on Windows and Linux, Cmd + ` on macOS). Create the environment the first time only:

```
python3 -m venv .venv      # macOS
python3 -m venv .venv      # Linux
py -m venv .venv           # Windows (PowerShell)
```

Then activate it — **every time you open a new terminal**:

```
source .venv/bin/activate  # macOS
source .venv/bin/activate  # Linux
.venv\Scripts\activate     # Windows (PowerShell)
```

You will see (.venv) at the start of the prompt. If you do not, nothing below will find pandas. Install the packages once, with the environment active:

```
pip install -r requirements.txt
```

Finally tell the editor which Python to use: Ctrl/Cmd + Shift + P, type **Python: Select Interpreter**, and choose the one inside .venv.

A new file, and running it

File > New File, then save it with a .py ending — session03.py, say. Write your code, save it, and in the terminal:

```
python session03.py        # macOS, Linux and Windows alike, inside .venv
```

Inside the environment the command is **python** on all three platforms. Outside it, macOS and Linux need **python3** and Windows needs **py**, which is one more reason to activate first.

For trying one line at a time, type **python** on its own to get the >>> prompt, and **exit()** to leave. The editor's **Run** triangle does the same thing as the command above.

The four things you always do

```
import pandas as pd
```

```
# 1 - build a DataFrame
df = pd.DataFrame({"country": ["FR", "DE", "IT"],
                   "gdp_pc_eur": [37_000, 46_000, 32_000]})
```

```
# 2 - look at what is in it
df.head()      # the first five rows
df.shape       # (rows, columns)
df.dtypes      # what type each column holds
```

```

df.info()          # types, and how many values are missing
df.describe()      # count, mean, sd and quartiles, per numeric column

# 3 - read a file in
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")          # needs openpyxl
df = pd.read_parquet("data/spine/core.parquet")

import qmib
df = qmib.load("core")                   # a course dataset, by name

# 4 - write a CSV out
df.to_csv("out.csv", index=False)      # index=False, or you get a stray column

```

From session 4

Load and shape

```
X = pd.get_dummies(d[["grade"]], drop_first=True)
```

Describe

```

pd.crosstab(d["grade"], d["default"])

d["default"].value_counts(normalize=True)    # how unbalanced is it

```

Fit

```

fit = smf.logit("default ~ fico + dti", data=d).fit()

from statsmodels.miscmodels.ordinal_model import OrderedModel
fit = OrderedModel(y, X, distr="logit").fit(method="bfgs")

import statsmodels.api as sm
fit = sm.MNLogit(y, sm.add_constant(X)).fit()

```

```
X = sm.add_constant(X)          # the intercept, with no formula
```

Read the fit

```

p = fit.predict(newdata)        # probabilities, not classes

fit.get_prediction(newdata).summary_frame()  # with an interval

np.exp(fit.params)              # a log-odds becomes an odds ratio

```

Choose and validate

```

lr = 2 * (full.llf - reduced.llf)  # the likelihood-ratio statistic

stats.chi2.sf(lr, df=2)            # its p-value

```

Plot

```
fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.9))
```